



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN

GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Miranda González José Francisco

N° de Cuenta: 318222327

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 04

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: 19/10/24

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Reporte de práctica 9: Animación Básica

Instrucciones:

1. Su dado de 10 caras cae sobre el piso, gira y cae en un número "random", se repite la tirada al presionar una tecla.
2. Por integrante del equipo elegirán un tipo de vehículo: terrestre o aéreo . Cada integrante del equipo creará un recorrido en el cual el vehículo se moverá alrededor de su tablero de Monopoly. Cada vehículo iniciará su recorrido a partir de una esquina diferente. (el vehículo terrestre no puede ser un carro o vehículo similar motorizado de 4 ruedas, se debe de tener movimiento de llantas o de hélices en sus vehículos.)

Actividades realizadas

Para completar las actividades solicitadas realice lo siguiente:

Para la actividad 1:

Como crear y texturizar el dado fue una actividad anterior a esta practica solo tuve que copiar y pegar lo necesario para mostrarlo en el nuevo proyecto.

En cuanto a los nuevos cambios dentro de main.cpp.

Encabezados que me ayudaron para elegir los grados que rotara el dado en cada eje de manera aleatoria.

```
49  // PARA ELEGIR LOS GRADOS DE MANERA RANDOM
50  ///////////////////////////////////////////////////////////////////
51  #include <iostream>
52  #include <cstdlib> // Para rand() y srand()
53  #include <ctime>   // Para time()
54  ///////////////////////////////////////////////////////////////////
```

Variables de animación del dado.

Para subir y bajar el dado y para rotar en cada eje.

```

76 // PRACTICA 09
77 // VARIABLES PARA ANIMACION
78 // DADO
79 ///////////////////////////////////////////////////
80 float subeBajaDado;
81 float subeBajaDadoOffset;
82
83 float rotaDadox;
84 float rotaDadoxOffset;
85 float rotaDadoy;
86 float rotaDadoyOffset;
87 float rotaDadoz;
88 float rotaDadozOffset;
89 ///////////////////////////////////////////////////

```

Valores de las variables de animación.

```

565 // PRACTICA 09
566 // DADO
567 ///////////////////////////////////////////////////
568 subeBajaDado = 20.0f;
569 subeBajaDadoOffset = 0.03;
570
571 rotaDadox = 0.0f;
572 rotaDadoxOffset = 1.0;
573
574 rotaDadoy = 0.0f;
575 rotaDadoyOffset = 1.0;
576
577 rotaDadoz = 0.0f;
578 rotaDadozOffset = 1.0;
579 ///////////////////////////////////////////////////

```

Elegir los grados que girara el dado en cada eje.

```

581 // PRACTICA 09
582 // GRADOS ALEATORIOS
583 ///////////////////////////////////////////////////
584
585 // Inicializar la semilla solo una vez
586 std::srand(static_cast<unsigned>(std::time(0)));
587
588 // PARA GRADOS DE Y
589 std::vector<int> listaY = { 180, -180 };
590 int gradosDeY = listaY[std::rand() % listaY.size()];
591
592 // PARA GRADOS DE Z
593 std::vector<int> listaZ = { 36, 108, 180, 252, 324, -36, -108, -180, -252, -324 };
594 int gradosDeZ = listaZ[std::rand() % listaZ.size()];
595
596 // PARA GRADOS DE X
597 std::vector<int> listaX = { 25, -25 };
598 int gradosDeX = listaX[std::rand() % listaX.size()];
599
600 ///////////////////////////////////////////////////

```

Estos grados se eligieron de la siguiente manera.

Inicialmente tenemos al dado de la siguiente forma:



Al bajar girara en el eje Y 180° o -180° , después, girara en el eje Z 36° , 108° , 180° , 252° , 324° , -36° , -108° , -180° , -252° o -324° y por último girara en X 25° o -25° .

Los grados de X se eligieron de esa manera ya que si colocamos un pentágono dentro de una circunferencia, la mitad de cada arista del pentágono tendrá el valor de los grados elegidos. Esto garantiza que una cara quedara hacia arriba.

En cuanto a los grados de X, esos 25° o -25° hacen que solo se levante una cara del dado, de lo contrario el dado quedaría como la imagen de arriba y no se podrá elegir un número.

Ahora pasando con la animación.

Al presionar la tecla Y el dado comenzará a bajar, una vez abajo implemente un switch que calcula todas las posibles combinaciones de los grados anteriores y en base a eso se decidirá el número del dado.

```
716 // PARA BAJAR
717 // Y
718 if (mainWindow.getsubeBajaDado() == true)
719 {
720     // HACER QUE EL DADO BAJE
721     if (subeBajaDado > 1)
722     {
723         subeBajaDado -= subeBajaOffset * deltaTime;
724     }
725     // CUANDO LLEGUE AL FINAL
726     else
727     {
728         // ROTAR EN Y
729         switch (gradosDeY)
730         {
731             case 180:
732                 if (rotaDadoy < 180)
733                 {
734                     rotaDadoy += rotaDadoyOffset * deltaTime;
735                 }
736                 else
737                 {
738                     // ROTAR EN Z
739                     switch (gradosDeZ)
740                     {
741                         case 36:
742                             if (rotaDadoz < 36)
743                             {
744                                 rotaDadoz += rotaDadozOffset * deltaTime;
745                             }
746                             else
747                             {
748                                 // ROTAR EN X
749                                 // ELEGIR EL NUMERO DEL LADO
750                                 switch (gradosDeX)
751                                 {
752                                     case 25:
753                                         if (rotaDadox < 25)
754                                         {
755                                             rotaDadox += rotaDadoxOffset * deltaTime;
756                                         }
```

```

757         break;
758     case -25:
759         if (rotaDadox > -25)
760         {
761             rotaDadox -= rotaDadoxOffset * deltaTime;
762         }
763         break;
764     }
765 }
766 break;
767 case -36:
768     if (rotaDadoz > -36)
769     {
770         rotaDadoz -= rotaDadozOffset * deltaTime;
771     }
772     else
773     {
774         switch (gradosDeX)
775         {
776             case 25:
777                 if (rotaDadox < 25)
778                 {
779                     rotaDadox += rotaDadoxOffset * deltaTime;
780                 }
781                 break;
782             case -25:
783                 if (rotaDadox > -25)
784                 {
785                     rotaDadox -= rotaDadoxOffset * deltaTime;
786                 }
787                 break;
788         }
789     }
790     break;
791 case 108:
792     if (rotaDadoz < 108)
793     {
794         rotaDadoz += rotaDadozOffset * deltaTime;
795     }
796     else
797     {
798         switch (gradosDeX)
799         {
800             case 25:
801                 if (rotaDadox < 25)

```

```

802         {
803             rotaDadox += rotaDadoxOffset * deltaTime;
804         }
805         break;
806     case -25:
807         if (rotaDadox > -25)
808         {
809             rotaDadox -= rotaDadoxOffset * deltaTime;
810         }
811         break;
812     }
813 }
814 break;
815 case -108:
816     if (rotaDadoz > -108)
817     {
818         rotaDadoz -= rotaDadozOffset * deltaTime;
819     }
820     else
821     {
822         switch (gradosDeX)
823         {
824             case 25:
825                 if (rotaDadox < 25)
826                 {
827                     rotaDadox += rotaDadoxOffset * deltaTime;
828                 }
829                 break;
830             case -25:
831                 if (rotaDadox > -25)
832                 {
833                     rotaDadox -= rotaDadoxOffset * deltaTime;
834                 }
835                 break;
836         }
837     }
838     break;
839 case 180:
840     if (rotaDadoz < 180)
841     {
842         rotaDadoz += rotaDadozOffset * deltaTime;
843     }
844     else
845     {
846         switch (gradosDeX)

```

```

847         {
848             case 25:
849                 if (rotaDadox < 25)
850                 {
851                     rotaDadox += rotaDadoxOffset * deltaTime;
852                 }
853                 break;
854             case -25:
855                 if (rotaDadox > -25)
856                 {
857                     rotaDadox -= rotaDadoxOffset * deltaTime;
858                 }
859                 break;
860             }
861         }
862         break;
863     case -180:
864         if (rotaDadoz > -180)
865         {
866             rotaDadoz -= rotaDadozOffset * deltaTime;
867         }
868         else
869         {
870             switch (gradosDeX)
871             {
872                 case 25:
873                     if (rotaDadox < 25)
874                     {
875                         rotaDadox += rotaDadoxOffset * deltaTime;
876                     }
877                     break;
878                 case -25:
879                     if (rotaDadox > -25)
880                     {
881                         rotaDadox -= rotaDadoxOffset * deltaTime;
882                     }
883                     break;
884             }
885         }
886         break;
887     case 252:
888         if (rotaDadoz < 252)
889         {
890             rotaDadoz += rotaDadozOffset * deltaTime;
891         }

```

```

891     }
892     else
893     {
894         switch (gradosDeX)
895         {
896             case 25:
897                 if (rotaDadox < 25)
898                 {
899                     rotaDadox += rotaDadoxOffset * deltaTime;
900                 }
901                 break;
902             case -25:
903                 if (rotaDadox > -25)
904                 {
905                     rotaDadox -= rotaDadoxOffset * deltaTime;
906                 }
907                 break;
908             }
909         }
910         break;
911     case -252:
912         if (rotaDadoz > -252)
913         {
914             rotaDadoz -= rotaDadozOffset * deltaTime;
915         }
916         else
917         {
918             switch (gradosDeX)
919             {
920                 case 25:
921                     if (rotaDadox < 25)
922                     {
923                         rotaDadox += rotaDadoxOffset * deltaTime;
924                     }
925                     break;
926                 case -25:
927                     if (rotaDadox > -25)
928                     {
929                         rotaDadox -= rotaDadoxOffset * deltaTime;
930                     }
931                     break;
932             }
933         }
934         break;
935     case 324:
936         if (rotaDadoz < 324)

```

```

937         {
938             rotaDadoz += rotaDadozOffset * deltaTime;
939         }
940     else
941     {
942         switch (gradosDeX)
943         {
944             case 25:
945                 if (rotaDadox < 25)
946                 {
947                     rotaDadox += rotaDadoxOffset * deltaTime;
948                 }
949                 break;
950             case -25:
951                 if (rotaDadox > -25)
952                 {
953                     rotaDadox -= rotaDadoxOffset * deltaTime;
954                 }
955                 break;
956             }
957         }
958         break;
959     case -324:
960         if (rotaDadoz > -324)
961         {
962             rotaDadoz -= rotaDadozOffset * deltaTime;
963         }
964     else
965     {
966         switch (gradosDeX)
967         {
968             case 25:
969                 if (rotaDadox < 25)
970                 {
971                     rotaDadox += rotaDadoxOffset * deltaTime;
972                 }
973                 break;
974             case -25:
975                 if (rotaDadox > -25)
976                 {
977                     rotaDadox -= rotaDadoxOffset * deltaTime;
978                 }
979                 break;
980             }
981         }

```

```

982         break;
983     }
984 }
985 break;
986 case -180:
987     if (rotaDadoz > -180)
988     {
989         rotaDadoz -= rotaDadozOffset * deltaTime;
990     }
991 else
992 {
993     switch (gradosDeZ)
994     {
995         case 36:
996             if (rotaDadoz < 36)
997             {
998                 rotaDadoz += rotaDadozOffset * deltaTime;
999             }
1000         else
1001         {
1002             switch (gradosDeX)
1003             {
1004                 case 25:
1005                     if (rotaDadox < 25)
1006                     {
1007                         rotaDadox += rotaDadoxOffset * deltaTime;
1008                     }
1009                     break;
1010                 case -25:
1011                     if (rotaDadox > -25)
1012                     {
1013                         rotaDadox -= rotaDadoxOffset * deltaTime;
1014                     }
1015                     break;
1016                 }
1017             }
1018             break;
1019         case -36:
1020             if (rotaDadoz > -36)
1021             {
1022                 rotaDadoz -= rotaDadozOffset * deltaTime;
1023             }
1024         else
1025         {
1026             switch (gradosDeX)

```

```

1027 {
1028     case 25:
1029         if (rotaDadox < 25)
1030         {
1031             rotaDadox += rotaDadoxOffset * deltaTime;
1032         }
1033         break;
1034     case -25:
1035         if (rotaDadox > -25)
1036         {
1037             rotaDadox -= rotaDadoxOffset * deltaTime;
1038         }
1039         break;
1040     }
1041 }
1042 break;
1043 case 108:
1044     if (rotaDadoz < 108)
1045     {
1046         rotaDadoz += rotaDadozOffset * deltaTime;
1047     }
1048     else
1049     {
1050         switch (gradosDeX)
1051         {
1052             case 25:
1053                 if (rotaDadox < 25)
1054                 {
1055                     rotaDadox += rotaDadoxOffset * deltaTime;
1056                 }
1057                 break;
1058             case -25:
1059                 if (rotaDadox > -25)
1060                 {
1061                     rotaDadox -= rotaDadoxOffset * deltaTime;
1062                 }
1063                 break;
1064             }
1065         }
1066     }
1067     break;
1068 case -108:
1069     if (rotaDadoz > -108)
1070     {
1071         rotaDadoz -= rotaDadozOffset * deltaTime;
1072     }

```

```

1072     else
1073     {
1074         switch (gradosDeX)
1075         {
1076             case 25:
1077                 if (rotaDadox < 25)
1078                 {
1079                     rotaDadox += rotaDadoxOffset * deltaTime;
1080                 }
1081                 break;
1082             case -25:
1083                 if (rotaDadox > -25)
1084                 {
1085                     rotaDadox -= rotaDadoxOffset * deltaTime;
1086                 }
1087                 break;
1088             }
1089         }
1090     }
1091     break;
1092 case 180:
1093     if (rotaDadoz < 180)
1094     {
1095         rotaDadoz += rotaDadozOffset * deltaTime;
1096     }
1097     else
1098     {
1099         switch (gradosDeX)
1100         {
1101             case 25:
1102                 if (rotaDadox < 25)
1103                 {
1104                     rotaDadox += rotaDadoxOffset * deltaTime;
1105                 }
1106                 break;
1107             case -25:
1108                 if (rotaDadox > -25)
1109                 {
1110                     rotaDadox -= rotaDadoxOffset * deltaTime;
1111                 }
1112                 break;
1113             }
1114         }
1115     }
1116     break;
1117 case -180:
1118     if (rotaDadoz > -180)

```



```

1117         {
1118             rotaDadoz -= rotaDadozOffset * deltaTime;
1119         }
1120         else
1121         {
1122             switch (gradosDeX)
1123             {
1124                 case 25:
1125                     if (rotaDadox < 25)
1126                     {
1127                         rotaDadox += rotaDadoxOffset * deltaTime;
1128                     }
1129                     break;
1130                 case -25:
1131                     if (rotaDadox > -25)
1132                     {
1133                         rotaDadox -= rotaDadoxOffset * deltaTime;
1134                     }
1135                     break;
1136             }
1137         }
1138         break;
1139     case 252:
1140         if (rotaDadoz < 252)
1141         {
1142             rotaDadoz += rotaDadozOffset * deltaTime;
1143         }
1144         else
1145         {
1146             switch (gradosDeX)
1147             {
1148                 case 25:
1149                     if (rotaDadox < 25)
1150                     {
1151                         rotaDadox += rotaDadoxOffset * deltaTime;
1152                     }
1153                     break;
1154                 case -25:
1155                     if (rotaDadox > -25)
1156                     {
1157                         rotaDadox -= rotaDadoxOffset * deltaTime;
1158                     }
1159                     break;
1160             }
1161         }

```

```

1162         break;
1163     case -252:
1164         if (rotaDadoz > -252)
1165         {
1166             rotaDadoz -= rotaDadozOffset * deltaTime;
1167         }
1168         else
1169         {
1170             switch (gradosDeX)
1171             {
1172                 case 25:
1173                     if (rotaDadox < 25)
1174                     {
1175                         rotaDadox += rotaDadoxOffset * deltaTime;
1176                     }
1177                     break;
1178                 case -25:
1179                     if (rotaDadox > -25)
1180                     {
1181                         rotaDadox -= rotaDadoxOffset * deltaTime;
1182                     }
1183                     break;
1184             }
1185         }
1186         break;
1187     case 324:
1188         if (rotaDadoz < 324)
1189         {
1190             rotaDadoz += rotaDadozOffset * deltaTime;
1191         }
1192         else
1193         {
1194             switch (gradosDeX)
1195             {
1196                 case 25:
1197                     if (rotaDadox < 25)
1198                     {
1199                         rotaDadox += rotaDadoxOffset * deltaTime;
1200                     }
1201                     break;
1202                 case -25:
1203                     if (rotaDadox > -25)
1204                     {
1205                         rotaDadox -= rotaDadoxOffset * deltaTime;
1206                     }

```

```

1207         break;
1208     }
1209 }
1210 break;
1211 case -324:
1212     if (rotaDadoz > -324)
1213     {
1214         rotaDadoz -= rotaDadozOffset * deltaTime;
1215     }
1216     else
1217     {
1218         switch (gradosDeX)
1219         {
1220             case 25:
1221                 if (rotaDadox < 25)
1222                 {
1223                     rotaDadox += rotaDadoxOffset * deltaTime;
1224                 }
1225                 break;
1226             case -25:
1227                 if (rotaDadox > -25)
1228                 {
1229                     rotaDadox -= rotaDadoxOffset * deltaTime;
1230                 }
1231                 break;
1232             }
1233         }
1234     }
1235 }
1236 }
1237 break;
1238 }
1239 }
1240 }

```

Después de decidir la cara del dado, con la tecla U este subirá y arriba volverá a su estado original, además se elegirán nuevos valores de los grados.

```

1242 // PARA SUBIR
1243 // U
1244 if (mainWindow.getsubeBajaDado() == false)
1245 {
1246     if (subeBajaDado < 20)
1247     {
1248         subeBajaDado += subeBajaOffset * deltaTime;
1249     }
1250     else
1251     {
1252         // REGRESAR A SU ESTADO ORIGINAL Y REASIGNAR LOS VALORES DE LOS GRADOS
1253         gradosDeY = listaY[std::rand() % listaY.size()];
1254         rotaDadoy = 0.0f;
1255         gradosDeZ = listaZ[std::rand() % listaZ.size()];
1256         rotaDadoz = 0.0f;
1257         gradosDeX = listaX[std::rand() % listaX.size()];
1258         rotaDadox = 0.0f;
1259     }
1260 }

```

Al terminar la animación dibuje el dado.

```

1427 // PRACTICA 09
1428 // DADO 10 CARAS
1429 ///////////////////////////////////////////////////////////////////
1430 model = glm::mat4(1.0);
1431 model = glm::translate(model, glm::vec3(40.0f, subeBajaDado, 15.0f));
1432 model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
1433 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
1434
1435 model = glm::rotate(model, rotaDadox * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
1436 model = glm::rotate(model, rotaDadoy * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
1437 model = glm::rotate(model, rotaDadoz * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
1438
1439 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1440 dado10Caras.UseTexture(); // CAMBIAMOS POR NUESTRA TEXTURA
1441 meshList[4]->RenderMesh();
1442 ///////////////////////////////////////////////////////////////////

```

Para bajar y subir el dado realice unos cambios en los archivos window.h y window.cpp.

```

17 // PRACTICA 09
18 ///////////////////////////////////////////////////////////////////
19 GLfloat getsubeBajaDado() { return subirBajarDado; }
20 ///////////////////////////////////////////////////////////////////

```

```

39 //PRACTICA 09
40 ///////////////////////////////////////////////////////////////////
41 GLfloat subirBajarDado;
42 ///////////////////////////////////////////////////////////////////

```

```

19 // PRACTICA 09
20 ///////////////////////////////////////////////////////////////////
21 subirBajarDado = false;
22 ///////////////////////////////////////////////////////////////////

```

```

114 // PRACTICA 09
115 ///////////////////////////////////////////////////////////////////
116 if (key == GLFW_KEY_Y)
117 {
118     theWindow->subirBajarDado = true;
119 }
120 if (key == GLFW_KEY_U)
121 {
122     theWindow->subirBajarDado = false;
123 }
124 ///////////////////////////////////////////////////////////////////

```

Para la actividad 2:

Primero cree las variables para la animación del vehículo.

```

91 // PRACTICA 09
92 // VARIABLES PARA ANIMACION
93 // VEHICULO
94 ///////////////////////////////////////////////////////////////////
95 float mueveVehiculo;
96 float mueveVehiculoOffset;

```

```

98     float mueveVehiculo2;
99     float mueveVehiculoOffset2;
100
101     float mueveVehiculo3;
102     float mueveVehiculoOffset3;
103
104     float mueveVehiculo4;
105     float mueveVehiculoOffset4;
106
107     float rotaHelice;
108     float rotaHeliceOffset;
109
110     float rotaVehiculo;
111     float rotaVehiculoOffset;
112
113     float rotaVehiculo2;
114     float rotaVehiculoOffset2;
115
116     float rotaVehiculo3;
117     float rotaVehiculoOffset3;
118
119     float rotaVehiculo4;
120     float rotaVehiculoOffset4;
121
122     bool comenzar;
123     //////////////////////////////////////

```

Cree las texturas.

```

146     // PRACTICA 09
147     // TEXTURAS
148     //////////////////////////////////////
149     Texture dado10Caras;
150     Texture H1;
151     Texture H2;
152     Texture H3;
153     Texture H4;
154     Texture H5;
155     Texture colorBlancoCasilla;
156     //////////////////////////////////////

```

Cree los modelos.

```

169     // PRACTICA 09
170     // MODELOS
171     //////////////////////////////////////
172     Model casilla;
173     Model helicoptero;
174     Model helices;
175     //////////////////////////////////////

```

Cargar las texturas.

```
447 // PRACTICA 09
448 // CAGAR TEXTURAS
449 //////////////////////////////////////////////////
450 dado10Caras = Texture("Textures/dado10CarasGIMP.png");
451 dado10Caras.LoadTextureA();
452 H1 = Texture("Textures/body.png");
453 H1.LoadTextureA();
454 H2 = Texture("Textures/P_body.png");
455 H2.LoadTextureA();
456 H3 = Texture("Textures/P_tail.png");
457 H3.LoadTextureA();
458 H4 = Texture("Textures/pely_a.png");
459 H4.LoadTextureA();
460 H5 = Texture("Textures/prope.png");
461 H5.LoadTextureA();
462 colorBlancoCasilla = Texture("Textures/colorBlanco.png");
463 colorBlancoCasilla.LoadTextureA();
464 //////////////////////////////////////////////////
```

Cargar los modelos.

```
482 // PRACTICA 09
483 // CAGAR MODELOS
484 //////////////////////////////////////////////////
485 casilla = Model();
486 casilla.LoadModel("Models/casilla.obj");
487 helicoptero = Model();
488 helicoptero.LoadModel("Models/helicoptero.obj");
489 helices = Model();
490 helices.LoadModel("Models/helices.obj");
491 //////////////////////////////////////////////////
```

Valores para las variables de animación.

```
602 // PRACTICA 09
603 // VEHICULO
604 //////////////////////////////////////////////////
605 mueveVehiculo = 0.0f;
606 mueveVehiculoOffset = 0.5f;
607
608 mueveVehiculo2 = 0.0f;
609 mueveVehiculoOffset2 = 0.5f;
610
611 mueveVehiculo3 = 0.0f;
612 mueveVehiculoOffset3 = 0.5f;
613
614 mueveVehiculo4 = 0.0f;
615 mueveVehiculoOffset4 = 0.5f;
```

```

617     rotaHelice = 0.0f;
618     rotaHeliceOffset = 2.0f;
619
620     rotaVehiculo = 0.0f;
621     rotaVehiculoOffset = 1.0f;
622
623     rotaVehiculo2 = 0.0f;
624     rotaVehiculoOffset2 = 1.0f;
625
626     rotaVehiculo3 = 0.0f;
627     rotaVehiculoOffset3 = 1.0f;
628
629     rotaVehiculo4 = 0.0f;
630     rotaVehiculoOffset4 = 1.0f;
631
632     comenzar = true;;
633     //////////////////////////////////

```

Después de todo lo anterior continúe con la animación.

Esta consistía en partir desde una esquina del “tablero” y al llegar al final rotar 90° y repetir.

```

1262 // VEHICULO
1263 if (glfwGetTime() > 5)
1264 {
1265     if (comenzar)
1266     {
1267         if (mueveVehiculo < 60)
1268         {
1269             mueveVehiculo += mueveVehiculoOffset * deltaTime;
1270             rotaHelice += rotaHeliceOffset * deltaTime;
1271         }
1272         else
1273         {
1274             if (rotaVehiculo > -90)
1275             {
1276                 rotaVehiculo -= rotaVehiculoOffset * deltaTime;
1277                 rotaHelice += rotaHeliceOffset * deltaTime;
1278             }
1279             else
1280             {
1281                 if (mueveVehiculo2 < 76)
1282                 {
1283                     mueveVehiculo2 += mueveVehiculoOffset2 * deltaTime;
1284                     rotaHelice += rotaHeliceOffset * deltaTime;
1285                 }
1286                 else
1287                 {
1288                     if (rotaVehiculo2 > -90)
1289                     {
1290                         rotaVehiculo2 -= rotaVehiculoOffset2 * deltaTime;
1291                         rotaHelice += rotaHeliceOffset * deltaTime;
1292                     }
1293                     else
1294                     {
1295                         if (mueveVehiculo3 < 60)
1296                         {
1297                             mueveVehiculo3 += mueveVehiculoOffset3 * deltaTime;
1298                             rotaHelice += rotaHeliceOffset * deltaTime;
1299                         }
1300                         else
1301                         {
1302                             if (rotaVehiculo3 > -90)
1303                             {
1304                                 rotaVehiculo3 -= rotaVehiculoOffset3 * deltaTime;

```

```

1305         rotaHelice += rotaHeliceOffset * deltaTime;
1306     }
1307     else
1308     {
1309         if (mueveVehiculo4 < 76)
1310         {
1311             mueveVehiculo4 += mueveVehiculoOffset4 * deltaTime;
1312             rotaHelice += rotaHeliceOffset * deltaTime;
1313         }
1314         else
1315         {
1316             if (rotaVehiculo4 > -90)
1317             {
1318                 rotaVehiculo4 -= rotaVehiculoOffset4 * deltaTime;
1319                 rotaHelice += rotaHeliceOffset * deltaTime;
1320             }
1321             else
1322             {
1323                 comenzar = !comenzar;
1324             }
1325         }
1326     }
1327 }
1328 }
1329 }
1330 }
1331 }
1332 }
1333 }
1334 else
1335 {
1336     // REINICAR VALORES
1337     mueveVehiculo = 0.0f;
1338     mueveVehiculo2 = 0.0f;
1339     mueveVehiculo3 = 0.0f;
1340     mueveVehiculo4 = 0.0f;
1341
1342     rotaVehiculo = 0.0f;
1343     rotaVehiculo2 = 0.0f;
1344     rotaVehiculo3 = 0.0f;
1345     rotaVehiculo4 = 0.0f;
1346
1347     comenzar = !comenzar;
1348 }
1349 }

```

Cuando se completa una vuelta se reinician los valores de las variables y de esta forma continua la animación.

Al terminar la animación, cree las matrices auxiliares para las casillas y el vehículo.

```

1399 // PRACTICA 09
1400 // MATRIZ AUXILIAR PARA CASILLAS
1401 ///////////////////////////////////////////////////
1402 glm::mat4 modelauxCasilla(1.0);
1403 ///////////////////////////////////////////////////
1404
1405 // PRACTICA 09
1406 // MATRIZ AUXILIAR PARA EL HELICOPTERO
1407 ///////////////////////////////////////////////////
1408 glm::mat4 modelauxHelicoptero(1.0);
1409 ///////////////////////////////////////////////////

```

Dibujo del tablero.

Ejecución del programa:

Vista inicial:



Dado:

Arriba.



Abajo al presionar Y (rotado).



Numero 6 en esa tirada.

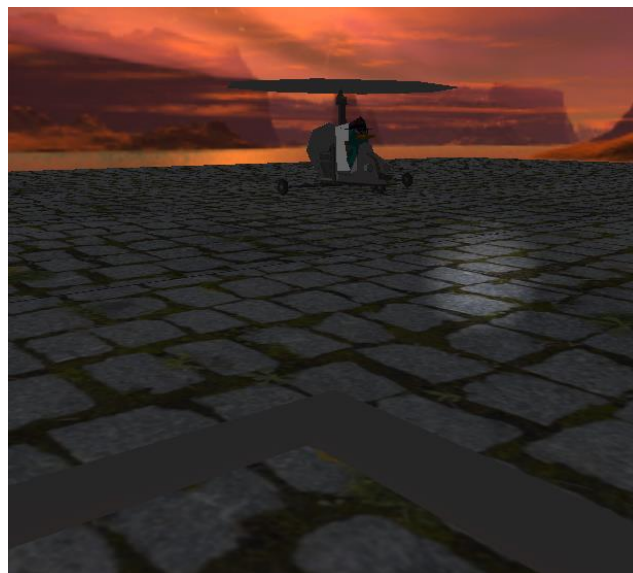


Arriba nuevamente al presionar U (estado original).

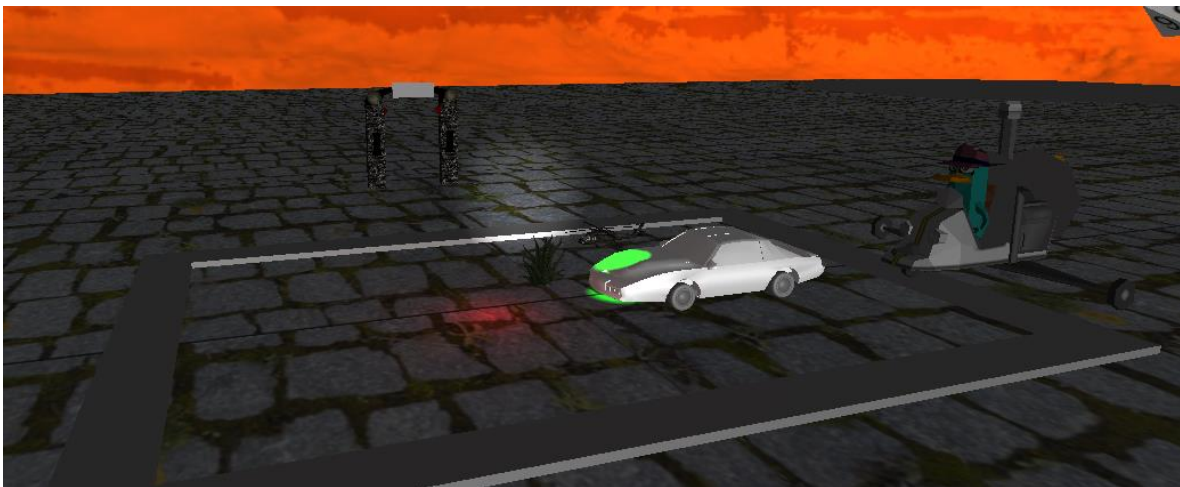


Vehículo:

En la esquina superior izquierda (mirando hacia -X).



Partes del recorrido por fuera del tablero.



Las animaciones no se pueden apreciar bien por medio de las imágenes.

En el video se podrá observar de manera correcta.

Problemas presentados

1. Complejidad

Crear las animaciones no fue tan difícil, pero si muy complejo.

En el vehículo no me lleve muchas líneas de código pero en el dado si, ya que el switch considera todos los casos posibles.

Todas las animaciones realizan los solicitado, pero haciendo uso de varias líneas de código que llegan a ser repetitivas.

2. Tablero

Improvise la forma de un tablero de Monopoly ya que aun no terminamos el de nuestro proyecto y en vez de usar el tiempo en terminarlo mejor me concentre en terminar la práctica. Pero después de realizar la animación ya no será tan difícil implementarla en el tablero del proyecto, solo necesitare ajustar los valores en las distancias de cada lado.

Conclusión

La practica fue entretenida pero muy compleja de realizar y esto es mas que nada debido al dado.

Realice una versión un poco mas pequeña de todas las condiciones del dado, la cual fue la siguiente:

```
// PARA BAJAR
// Y
if (mainWindow.getsubeBajaDado() == true)
{
    // HACER QUE EL DADO BAJE
    if (subeBajaDado > 1)
    {
        subeBajaDado -= subeBajaOffset * deltaTime;
    }
    // CUANDO LLEGUE AL FINAL
    else
    {
        // ROTAR EN Y
        if (gradosDeY > 0)
        {
            if (rotaDadoY < gradosDeY)
            {
                rotaDadoY += rotaDadoYOffset * deltaTime;
            }
            else
            {
                // ROTAR EN Z
                if (gradosDeZ > 0)
                {
                    if (rotaDadoZ < gradosDeZ)
                    {
                        rotaDadoZ += rotaDadoZOffset * deltaTime;
                    }
                    else
                    {
                        if (gradosDeX > 0)
                        {
                            // ROTAR EN X
```

```

// ELEGIR EL NUMERO DEL LADO
if (rotaDadox < gradosDeX)
{
    rotaDadox += rotaDadoxOffset *
deltaTime;
}
}
else
{
    if (rotaDadox > gradosDeX)
    {
        rotaDadox -= rotaDadoxOffset *
deltaTime;
    }
}
}
}
else
{
    if (rotaDadoz > gradosDeZ)
    {
        rotaDadoz -= rotaDadozOffset * deltaTime;
    }
    else
    {
        if (gradosDeX > 0)
        {
            if (rotaDadox < gradosDeX)
            {
                rotaDadox += rotaDadoxOffset *
deltaTime;
            }
            else
            {
                if (rotaDadox > gradosDeX)
                {
                    rotaDadox -= rotaDadoxOffset *
deltaTime;
                }
            }
        }
    }
}
}
}
}
else
{

```

```

        if (rotaDadoy > gradosDeY)
        {
            rotaDadoy -= rotaDadoyOffset * deltaTime;
        }
        else
        {
            if (gradosDeZ > 0)
            {
                if (rotaDadoz < gradosDeZ)
                {
                    rotaDadoz += rotaDadozOffset * deltaTime;
                }
                else
                {
                    if (gradosDeX > 0)
                    {
                        if (rotaDadox < gradosDeX)
                        {
                            rotaDadox += rotaDadoxOffset *
deltaTime;
                        }
                    }
                    else
                    {
                        if (rotaDadox > gradosDeX)
                        {
                            rotaDadox -= rotaDadoxOffset *
deltaTime;
                        }
                    }
                }
            }
        }
        else
        {
            if (rotaDadoz > gradosDeZ)
            {
                rotaDadoz -= rotaDadozOffset * deltaTime;
            }
            else
            {
                if (gradosDeX > 0)
                {
                    if (rotaDadox < gradosDeX)
                    {
                        rotaDadox += rotaDadoxOffset *
deltaTime;
                    }
                }
            }
        }
    }
}

```

```

    }
    else
    {
        if (rotaDadox > gradosDeX)
        {
            rotaDadox -= rotaDadoxOffset *
deltaTime;
        }
    }
}

```

Funcionaba de manera correcta, pero decidí dejar el switch pensando mas que nada en el proyecto, porque como separamos cada caso de manera muy específica ahí puedo asignar una variable que almacenara el resultado del dado.

También pensé en realizar funciones, pero volver a aprender como ocuparlas me iba a llevar tiempo, además que eso se vera en la siguiente practica y me imagino que reducirá demasiado las líneas de código.

Bibliografía

The Models Resource. (s.f). Perry's Helicopter. Consultado el 19 de octubre del 2024 de https://www.models-resource.com/ds_dsi/phineasandferbacrosstheseconddimension/model/14451/